

CIF Microarchitecture Specification

Read & Write Path

Overview

The Controller Interface (CIF) is the AXI-facing front-end of the memory subsystem. It translates AXI4 read and write transactions into normalized, fixed-width packets suitable for MC Core processing. The CIF handles burst splitting, transaction tagging, packet formation, response tracking, and in-order AXI response generation. MC Core remains transaction-agnostic and operates purely on a per-packet basis.

Global Assumptions

- AXI4 protocol.
- Data width: 512 bits (64 bytes). Parameterizable via RTL.
- Address width: 32 bits (FPGA initial configuration). Parameterizable.
- AXID width: 6 bits (64 unique AXI IDs).
- Supported burst types: INCR and WRAP. FIXED not supported.
- Multiple outstanding transactions supported per AXID (up to 4).
- Maximum burst length: 256 beats (AXI4 maximum).
- Narrow INCR and unaligned INCR transfers supported.
- Narrow WRAP and unaligned WRAP transfers not supported.
- Out-of-order (OoO) completions from MC Core are supported and resolved within CIF.
- Protocol violations (illegal burst type, unaligned WRAP, etc.) are design-time assertions only. Runtime validation is limited to address legality checks.

Parameters

Parameter	Default	Description
AXID_W	6	AXI ID width. Supports up to $2^6 = 64$ unique IDs.
ADDR_W	32	Address width in bits.
DATA_W	512	Data path width in bits.
N_AR	128	AR Metadata FIFO depth ($= 4 \times \text{N_CLIENTS}$).
N_AW	128	AW Metadata FIFO depth ($= 4 \times \text{N_CLIENTS}$).
N_W	64	W Data FIFO depth.
N_PKT	64	Packet FIFO depth (read and write).
MC_RD_BUF	64	MC Core internal read buffer depth. Defines ROB depth.
ROB_WATERMARK	26	MC-side watermark for backpressure signalling.
MAX_OUTSTANDING	4	Maximum outstanding transactions per AXID.
N_CLIENTS	32	AXI client count. Fixed at compile time (max 32).

All parameters are configurable at synthesis time.

Table 1: CIF Global Parameters

Note on N_CLIENTS. The number of AXI clients is fixed at synthesis time. Each client is statically allocated `MAX_OUTSTANDING = 4` slots in the AR and AW metadata FIFOs, giving `N_AR = N_AW = 4 × N_CLIENTS`. Idle clients retain their allocated slots; dynamic reallocation across clients is deferred to a future CIF revision.

Write Path

Overview

The write path accepts AXI write transactions on the AW and W channels, validates them, splits bursts into normalized 64-byte packets, and forwards them to MC Core. Per-packet completions are tracked and aggregated to generate a single AXI B channel response per transaction.

Block Descriptions

AW Metadata FIFO (depth = N_AW = 4 × N_CLIENTS). Captures and buffers incoming AW channel metadata: AWADDR, AWLEN, AWSIZE, AWBURST, AWID. Decouples AXI request arrival from downstream validation and scheduling. Each AXI client is statically allocated 4 slots. AWREADY is driven low when no slots are available for the requesting client.

W Data FIFO (depth = N_W). Captures and buffers AXI write data beats: WDATA, WSTRB, WLAST. Operates independently of the AW channel. W beats flow in freely regardless of AW validation status.

Request Validator. Pops entries from the AW Metadata FIFO and performs address legality checks combinationally:

- Address range validation.

Protocol-level constraints (burst type, WRAP legality, alignment, burst length) are enforced as design-time assertions and are not checked at runtime — compliant AXI masters are assumed. Valid requests are forwarded to the Transaction Tag Allocator. Address errors are forwarded to the Error Handler along with AWID.

Transaction Tag Allocator. Shared across all N_CLIENTS AXI clients. Allocates a unique `txn_tag` per validated transaction:

```
txn_tag[7:0] = AXID[5:0] + seqnum[1:0]
seqnum: monotonically increasing per AXID, wraps at 4
```

Accepts inputs from both the Request Validator (valid requests) and the Error Handler (NOP transactions). Seqnum allocation is blocked when all 4 outstanding slots for an AXID are occupied. Tags are freed when the Response Tracker emits a BRESP for the corresponding transaction.

Burst Splitter (two-stage pipeline). Shared across all N_CLIENTS AXI clients. Converts validated AXI burst requests into normalized beat-level packets.

- *Stage 1 — Row Boundary Split:* Splits bursts such that no sub-transaction crosses a DRAM row boundary. Generates row-confined sub-transactions.
- *Stage 2 — Command Alignment & Packetization:* Aligns packets to memory-command boundaries. Generates byte-enable wrapper masks for partial or edge-aligned packets.

Internal state: `base_addr`, `beat_idx`, `beats_left`, `burst_type`, `burst_len`, `txn_tag`.

Packet Formation Unit. Assembles normalized 618-bit write packets:

<code>pkt_type</code>	<code>[1:0]</code>	2 bits	(00=Write, 01=Read, 10=Err_write, 11=Err_read)
<code>txn_tag</code>	<code>[9:2]</code>	8 bits	
<code>addr</code>	<code>[41:10]</code>	32 bits	
<code>data</code>	<code>[553:42]</code>	512 bits	
<code>wrapper</code>	<code>[617:554]</code>	64 bits	(1 bit = 1 byte validity)
Total:		618 bits	

One AXI beat maps directly to one normalized packet. No beat accumulation required.

Packet FIFO (depth = N_PKT). Buffers normalized write packets before transfer to MC Core. Provides decoupling between CIF packet generation rate and MC Core processing rate.

Response Tracker. Shared across all N_CLIENTS AXI clients. Tracks outstanding write transactions on a per-tag basis. Stores: `txn_tag`, `AXID` (recovered as `txn_tag[7:2]`), `expected_packets` (from `AWLEN+1`), `completed_packets`, `status`. Emits a single `BRESP` when:

```
completed_packets == expected_packets
```

On receiving an error status from MC Core (corresponding to an `Err.write` packet), passes `DECERR` to the B Response Generator.

B Response Generator. Drives the AXI B channel: `BVALID`, `BRESP`, `BID`. `BID` is recovered from `txn_tag[7:2]`. Generates `DECERR` when an error status is received from the Response Tracker, and `OKAY` otherwise. Single path to the AXI B channel — no arbitration mux required.

Error Handler. Receives address error notifications from the Request Validator. Sets `pkt_type = Err.write` and forwards the transaction into the normal pipeline via the Transaction Tag Allocator. W beats for the transaction flow through the pipeline without draining or discarding. MC Core receives the `Err.write` packet and returns an error completion; `DECERR` is subsequently generated by the B Response Generator on the AXI B channel.

Write Packet Format

Field	Width (bits)	Description
<code>pkt_type</code>	2	Packet type: 00=Write, 01=Read, 10=Err.write, 11=Err.read
<code>txn_tag</code>	8	Transaction tag (<code>AXID[5:0] + seqnum[1:0]</code>)
<code>addr</code>	32	Write address
<code>data</code>	512	Write data payload
<code>wrapper</code>	64	Byte-level validity mask (1 bit = 1 byte)
Total	618	PKT_DATA[617:0]

Table 2: Write Packet Format

Write Path Backpressure Chain

Backpressure propagates right-to-left through the write pipeline:

```
MC Core busy -> PKT_READY = 0
  -> Packet FIFO fills up
  -> Packet Formation Unit stalls
  -> Burst Splitter stalls
  -> W Data FIFO absorbs backpressure
  -> W Data FIFO full -> WREADY = 0
  -> AW Metadata FIFO full -> AWREADY = 0
```

This avoids materializing large bursts upfront and keeps buffering requirements bounded.

Read Path

Overview

The read path accepts AXI read transactions on the AR channel, validates them, splits bursts into normalized read request packets, forwards them to MC Core, and reassembles in-order read

data for AXI R channel delivery. MC Core may return completions out of order; the CIF Reorder Buffer restores per-AXID AXI ordering before response generation.

Block Descriptions

AR Metadata FIFO (depth = $N_AR = 4 \times N_CLIENTS$). Captures and buffers incoming AR channel metadata: ARADDR, ARLEN, ARSIZE, ARBURST, ARID. Decouples AXI request arrival from downstream processing. Each AXI client is statically allocated 4 slots. ARREADY is driven low when no slots are available for the requesting client.

Request Validator. Performs address legality checks only. Protocol-level constraints are enforced as design-time assertions and are not checked at runtime. Valid requests proceed to the Transaction Tag Allocator. Address errors are forwarded to the Error Handler with ARID preserved.

Transaction Tag Allocator. Shared across all $N_CLIENTS$ AXI clients. Same encoding as the write path:

```
txn_tag[7:0] = AXID[5:0] + seqnum[1:0]
```

Accepts inputs from both the Request Validator (valid requests) and the Error Handler (NOP transactions). Seqnum is monotonically increasing per AXID and wraps at 4.

Burst Splitter. Shared across all $N_CLIENTS$ AXI clients. Same two-stage pipeline as write path. Generates per-beat read request contexts containing `txn_tag` and `addr`.

Packet Formation Unit. Assembles normalized 42-bit read request packets:

pkt_type	[1:0]	2 bits	(00=Write, 01=Read, 10=Err_write, 11=Err_read)
txn_tag	[9:2]	8 bits	
addr	[41:10]	32 bits	
Total:	42 bits		PKT_DATA[41:0]

Packet FIFO (depth = N_PKT). Buffers normalized read request packets before forwarding to MC Core.

Read Transaction Table (RTT). Allocated at tag assignment time. Stores per-transaction metadata:

- AXID
- expected_packets (from ARLEN+1)
- completed_packets

Asserts `rob_last` when `completed_packets == expected_packets`, signalling the final beat of a transaction to Merge Logic. Entry is freed after `rob_last` is emitted.

Reorder Buffer (ROB). Stores out-of-order read completions from MC Core and emits them in-order per AXID. Depth is tied to `MC_RD_BUF` — the ROB never needs more slots than MC Core can have in-flight simultaneously.

Structure:

```
ROB[MC_RD_BUF] -- flat buffer, depth = 64
Each slot:
    valid      1 bit
    data       512 bits
    status     4 bits
```

```
Free list: 64 entries
Lookup table: (AXID, seqnum) -> slot index
```

On tag allocation:

```
pop free slot index from free list
store mapping: (AXID, seqnum) -> slot index in lookup table
```

On MC completion:

```
extract AXID = tag[7:2], seqnum = tag[1:0]
lookup slot index from (AXID, seqnum)
ROB[slot].data    = completion data
ROB[slot].status  = completion status
ROB[slot].valid   = 1

emit loop:
    while ROB[slot_at_head].valid == 1:
        emit to Merge Logic
        ROB[slot_at_head].valid = 0
        push slot back to free list
        advance head pointer for AXID
```

Depth: MC_RD_BUF = 64. Implemented as dual-port SRAM/BRAM. Watermark threshold: MC_RD_BUF - ROB_WATERMARK = 64 - 26 = 38.

Merge Logic. Receives in-order beats from the ROB and performs per-ID beat sequencing. Streams ordered RDATA to the R Response Generator.

R Response Generator. AXI R channel driver. Restores RID from `txn_tag[7:2]`, generates RLAST, RRESP, asserts RVALID, and drives RDATA. Generates DECERR when an error status is received from the ROB, and OKAY otherwise. Single path to the AXI R channel — no arbitration mux required.

Error Handler. Receives address error notifications from the Request Validator with ARID preserved. Sets `pkt_type = Err_read` and forwards the transaction into the normal pipeline via the Transaction Tag Allocator. MC Core receives the `Err_read` packet and returns an error completion; DECERR is subsequently generated by the R Response Generator on the AXI R channel.

Read Packet Format

Field	Width (bits)	Description
<code>pkt_type</code>	2	Packet type: 00=Write, 01=Read, 10=Err_write, 11=Err_read
<code>txn_tag</code>	8	Transaction tag (AXID[5:0] + seqnum[1:0])
<code>addr</code>	32	Read address
Total	42	PKT_DATA[41:0]

Table 3: Read Packet Format

Read Path Backpressure Chain

MC Core does not support backpressure on the completion interface (`RESP_READY` must always be 1). CIF must therefore ensure the ROB never fills completely. Backpressure is applied by halting new read requests to MC Core when the ROB occupancy reaches the watermark threshold:

```

ROB occupancy >= MC_RD_BUF - ROB_WATERMARK  (= 64 - 26 = 38)
-> PKT_READY = 0  (stop sending new requests to MC)
-> Packet FIFO fills up
-> Burst Splitter stalls
-> AR Metadata FIFO absorbs backpressure
-> AR Metadata FIFO full -> ARREADY = 0

```

The ROB_WATERMARK = 26 headroom ensures all in-flight requests already accepted by MC Core can return completions without overflowing the ROB.

CIF ↔ MC Core Interface

Write Interface (CIF → MC Core)

Signal	Width	Description
PKT_VALID	1	Packet valid
PKT_READY	1	MC Core ready to accept
PKT_DATA	618	Serialized write packet

Table 4: Write Request Interface

Read Interface (CIF → MC Core)

Signal	Width	Description
PKT_VALID	1	Packet valid
PKT_READY	1	MC Core ready to accept
PKT_DATA	42	Serialized read request packet

Table 5: Read Request Interface

Completion Interface (MC Core → CIF)

Agreed with MC Core team. RESP_READY must be held high at all times by CIF — MC Core does not support backpressure on the completion path.

Signal	Width	Description
RESP_VALID	1	Completion valid
RESP_READY	1	Must always be 1 (CIF always ready)
resp_type	2	00=RD_DATA, 01=WR_ACK, 10=ERR
tag	8	tag[7:2]=AXID, tag[1:0]=seqnum
data	512	Read data (valid for RD_DATA only)
status	4	Status/error code (encoding TBD with MC team)

Table 6: MC Core Completion Interface — agreed with MC Core team

Packet Type Encoding

The pkt_type field is carried in bits [1:0] of all CIF-to-MC-Core packets. Agreed with MC Core team.

```

pkt_type[1:0]:
  00 = Write      -- normal write transaction
  01 = Read       -- normal read transaction
  10 = Err_write  -- address-error write; MC Core returns error
                  completion
  11 = Err_read   -- address-error read;  MC Core returns error
                  completion

```

MC Core discards data for `Err_write` and `Err_read` packets and returns an error status on the completion interface. CIF maps this to `DECERR` on the AXI B or R channel respectively.

Tag Encoding

```

txn_tag[7:0]:
  [7:2] = AXID    (6 bits, up to 64 unique IDs)
  [1:0] = seqnum  (2 bits, 0-3, per-AXID outstanding slot)

```

Agreed with MC Core team. MC Core returns the same `tag` value on completion that CIF embedded in the original request packet.

Edge Cases and Design Decisions

Invalid Request Handling — Write Path

The Request Validator performs address legality checks only at runtime. On an address error, the Error Handler sets `pkt_type = Err_write` and injects the transaction into the normal pipeline via the Transaction Tag Allocator. W beats for the transaction flow through the W Data FIFO and Packet Formation Unit without modification. MC Core receives the `Err_write` packet, discards the data, and returns an error completion. The B Response Generator drives `BRESP = DECERR` on the AXI B channel. No W beat draining or direct error bypass path is required.

Invalid Request Handling — Read Path

On an address error, the Error Handler sets `pkt_type = Err_read` and injects the transaction into the normal pipeline. MC Core receives the `Err_read` packet and returns an error completion. The R Response Generator drives `RRESP = DECERR` on the AXI R channel. No ROB slot is bypassed and no special read-path handling is required.

Protocol Violation Handling

Burst type violations, WRAP illegality, alignment errors, and maximum burst length violations are not checked at runtime. Compliant AXI4 masters are assumed. These constraints are enforced as design-time assertions in the verification environment only.

Error Response Path

Both read and write paths use a single response path — the normal Response Generator drives the AXI R and B channels respectively. The arbitration mux previously used to multiplex error and normal responses has been removed. All responses, including `DECERR`, are generated by the Response Generator based on the completion status received from MC Core.

Out-of-Order Completion Handling

MC Core may return read completions out of order. The ROB absorbs OoO completions using a flat buffer indexed by dynamically allocated slot. On completion, CIF resolves the slot index via a lookup table keyed on (AXID, seqnum). In-order emission per AXID is maintained by tracking a head pointer per AXID; a slot is emitted and freed only when the head slot for that AXID is valid. This guarantees AXI R channel ordering compliance.

ROB Sizing and Overflow Prevention

The ROB depth is tied to MC_RD_BUF — the MC Core internal read buffer depth. The ROB never needs more entries than MC Core can have in-flight simultaneously. With MC_RD_BUF = 64, the ROB is 64 entries deep. Since RESP_READY must always be 1, CIF cannot stall incoming completions. Instead, CIF halts new read request issuance (PKT_READY = 0) when ROB occupancy reaches MC_RD_BUF - ROB_WATERMARK = 38. The ROB_WATERMARK = 26 headroom absorbs all completions in-flight within MC Core without ROB overflow.

Seqnum Ordering Guarantee

The Transaction Tag Allocator guarantees monotonically increasing seqnums per AXID. This property underpins the correctness of the ROB head-pointer ordering scheme. Seqnum allocation stalls if all 4 outstanding slots for an AXID are occupied, providing implicit backpressure on the request path.

N_CLIENTS Scalability

N_CLIENTS is a compile-time parameter. AR and AW metadata FIFO depths scale as $4 \times N_CLIENTS$, ensuring each client always has 4 guaranteed outstanding slots. Idle clients retain their allocated slots; unutilized capacity is not reclaimed dynamically in this revision. A future CIF revision will introduce a shared slot pool — with 2 guaranteed slots per client and the remaining ($total_depth - 2 \times N_CLIENTS$) slots dynamically allocated — controlled by a QoS-aware arbitration fabric.